



Caruca v2

LLM-based specification mining for opaque shell commands

Evaluation Gaps — v1 / Paper

Fourteen opportunities to broaden testing

SOURCE `ai_docs/analysis/evaluation_gaps.md`

GENERATED August 29, 2026 at 7:08 PM

PAGES 11

CARUCA V2 — RESEARCH PROJECT

Table of contents

Evaluation Gaps — v1 / Paper

Part 1 — Gaps not identified or flagged by the authors

1. Multiple file arguments are varied independently, never in relation to each other [NOVEL]
2. Parallelizability is inferred by comparing sequential runs, not by running invocations concurrently [NOVEL]
3. The sandboxing/tracing stack is Linux-specific, while the paper’s own motivation cites cross-platform differences [NOVEL]
4. The real-world usage corpus is scoped to GitHub shell scripts, which is one slice of “real world” [NOVEL]
5. Directories capped at one level, one child [NOVEL]
6. No permission/ownership axis in the filesystem model [NOVEL]
7. File content fixtures don’t include structured/office document formats [NOVEL]
8. No empty/zero-byte content variant [NOVEL]
9. LLM syntax-generation is evaluated once per command, without a reproducibility/variance check [NOVEL]

Part 2 — Gaps already flagged by the authors

10. Symlinks are not part of the generated filesystem fixtures [PAPER-NOTED]
11. Pipe / early-reader-exit behavior (SIGPIPE) is untested [PAPER-NOTED]
12. Numeric arguments are only ever -1, 0, or 1 [SELF-ASSESSED]
13. The 97.78% coverage figure blends normalization layers of different strength [SELF-ASSESSED]
14. No environment-variable modeling anywhere in the generated invocation configurations [PAPER-NOTED]

Summary table

Evaluation Gaps — v1 / Paper

A running list of **opportunities to broaden test/evaluation coverage** in the v1 baseline (and in the paper’s evaluation of it) — places where the specification-mining pipeline could be extended to exercise a richer slice of real command behavior. Everything here is about *scope*, not correctness — sound engineering and research trade-offs made to keep an ambitious system tractable, not defects. None are labeled “egregious”; where a citation shows the authors already flagged something, that’s noted so credit is given. Consistent with this repo’s v1-vs-v2 framing (see `CLAUDE.md`): these are read as **extension opportunities for v2**, not as criticism of v1.

Paper cited as **Lamprou et al., “Caruca: Effective and Efficient Specification Mining for Opaque Software Components,” arXiv:2510.14279** (Oct 2025) — full text at `ai_docs/refs/caruca_white_paper.md`. Code citations are against v1, `~/stevens/caruca/` (git root; remote `binpash/caruca`), commit `d80324073`, branch `caruca-experiments`. See also `~/stevens/caruca/caruca/CODE_INSIGHTS.md` for v1 implementation bugs (a distinct, complementary list — that file is about correctness, this one is about coverage/scope).

Ordering: Part 1 lists gaps the authors do not appear to discuss anywhere in the paper. Part 2 lists gaps the authors already raise themselves — either as an explicit one-line acknowledgment of future work, or as a caveat reported alongside a measured result. Within each part, items are ordered roughly by how directly they bear on the paper’s central claims.

Status tags: **[NOVEL]** not discussed in the paper · **[PAPER-NOTED]** the authors flag the gap themselves, usually as future work · **[SELF-ASSESSED]** the paper measures this and reports the caveat, but the caveat is easy to lose in a headline number.

Part 1 — Gaps not identified or flagged by the authors

1. Multiple file arguments are varied independently, never in relation to each other [NOVEL]

`Path.attach_env ( ~/stevens/caruca/caruca/src/caruca/ir/string.py:91-123 )` and `ArgumentSequence.environment_variation ( ir/string.py:144-152 )` generate content for each argument position independently, then combine positions via `itertools.product ( ir/string.py:172 )`. This means two `Path` arguments in the same invocation are never deliberately made *identical*, *sorted-vs-unsorted*, or *one-a-prefix-of-the-other* — properties that commands like `diff`, `comm`, `join`, and `sort -c` are specifically designed around.

Opportunity: for multi- `Path` invocations, add a small set of explicitly correlated content pairs (identical, disjoint, one-is-subset) as an additional generation strategy layered on top of the existing independent product. **v2 relevance:** a naive-LLM or agentic approach could plausibly infer “this command compares two inputs” from its documentation and generate correlated fixtures without this being hand-coded — a concrete axis for the coverage comparison dimension in `CLAUDE.md`.

2. Parallelizability is inferred by comparing sequential runs, not by running invocations concurrently [NOVEL]

Per §6.3 (p.8), the stateless/pure classification compares the trace of one full invocation against the *concatenated* traces of running each split independently — but tracing happens one invocation configuration at a time (§5, p.7: “*it iterates over each invocation configuration... and executes the concrete command invocation*”). This is a reasonable and efficient way to establish the *output-equivalence* half of parallelizability. It does mean, though, that the inference never observes what happens when split invocations of the *same* command actually run **at the same time** — which is precisely how PaSh would run them in production. Race conditions on shared temp files, lock files, or PID-derived paths wouldn’t surface in sequential tracing.

Opportunity: as a validation pass (not necessarily part of the core inference), spot-check commands classified as parallelizable by actually launching their split invocations concurrently and diffing the result against the sequential prediction.

3. The sandboxing/tracing stack is Linux-specific, while the paper’s own motivation cites cross-platform differences [NOVEL]

The Introduction motivates the problem partly with “*differences across command versions on Linux, macOS, BSD, etc.*” (§1, p.2) as a reason handwritten specs don’t scale. But the tracer relies on OverlayFS, `strace`, Linux namespaces, and seccomp-BPF (§5, p.7) — all Linux-only mechanisms — and the evaluation (§7) is entirely GNU coreutils/POSIX/Shellcheck/Shseer, implicitly on Linux. The paper never demonstrates or measures whether the *same generated specifications* hold on macOS or BSD variants of these commands, so the cross-platform motivation and the evaluation’s scope don’t quite meet.

Opportunity: even a small case study — running the generated invocations against BSD/macOS builds of a handful of coreutils inside a VM or CI matrix — would let the paper’s motivating claim and its evidence line up directly. **v2 relevance:** directly relevant to the “isolation backend” question already open in `CLAUDE.md` (Firecracker/gVisor/bubblewrap candidates) — worth deciding whether v2’s isolation layer should be portable from the outset.

4. The real-world usage corpus is scoped to GitHub shell scripts, which is one slice of “real world” [NOVEL]

The invocation-frequency study underlying the flag-count pruning (§4.1, p.5) and the comprehensiveness evaluation (§7.3, p.11) both draw from the same source: “*repositories with more than 10 stars that use the shell as their primary language*” (§4.1, p.5) — the same corpus as `~/stevens/caruca/eval/command-invocations.txt`. That’s a solid, reproducible corpus, but it skews toward scripts written *as* shell scripts. A large amount of real-world command usage lives elsewhere — GitHub Actions `run:` blocks, Dockerfile `RUN` lines, cron entries, Makefile recipes — which may favor different flags/commands (e.g., heavier use of `apt-get`, `curl`, container-oriented flags) than standalone `.sh` files with >10 stars.

Opportunity: sampling a small additional corpus from CI/CD YAML or Dockerfiles would let the comprehensiveness number in §7.3 speak to a broader definition of “real-world usage,” complementing rather than replacing the existing study.

5. Directories capped at one level, one child [NOVEL]

`NonEmptyDirectory.prepare_env` (`ir/environment.py:235-242`) creates exactly one directory containing exactly one file named `file` ; there is no recursive tree-building anywhere in the codebase (only two `os.mkdir` calls exist total, neither in a loop). Interestingly, the paper's own formal model (Fig. 5, p.6) defines `FileSystem` *recursively* — `directory(List<FileSystem>) | file` — which could represent arbitrarily nested trees, but the text describing generation narrows this to “a directory (empty or containing one child)” (§4.3, p.6). This isn't discussed as a scoping decision anywhere in the paper.

Opportunity: since `rm -r`, `cp -r`, `chmod -R`, and `du` are all commands in the paper's own evaluation set and have documented recursive semantics, generating at least a 2-3 level tree variant would let the existing model's own recursive type be exercised as designed, and would give recursive flags a case that actually distinguishes “handles one file” from “correctly recurses.”

6. No permission/ownership axis in the filesystem model [NOVEL]

Fig. 5's model varies path type (file/directory/nonexistent) and string form (absolute/relative), but has no dimension for permission bits, ownership, or a read-only parent directory. Commands whose entire documented purpose is permissions (`chmod`, `chown`, `chgrp`, `stat`) or whose behavior branches on write access (`rm` on a read-only directory, `touch` on a file you don't own) don't get a fixture that varies along the axis they actually care about.

Opportunity: a small permission-state dimension (owner-writable / read-only / no-execute) alongside the existing type dimension, applied selectively to permission-relevant commands.

7. File content fixtures don't include structured/office document formats [NOVEL]

Content generation (`ir/contents.py:11-54`) offers five fixed byte blobs: plain text, “math” text, “time” text, one JSON file, and one JPEG. The paper matches this closely, describing “four types of content: text, math formulas, JSON, and image data” (§4.3, p.6) — the repo's extra `TIME` category is a small drift from the published description worth being aware of when citing the two together, but not itself consequential. Neither the paper nor the code includes Markdown, CSV, spreadsheet (`.xlsx`), presentation (`.pptx`), or database file content — and filenames only carry an extension for the `IMAGE` case (`ir/environment.py:139-143`), so extension-sensitive dispatch is untested too. Notably, `pandoc` and `unzip` — both format-conversion tools — are commands the paper's own evaluation discusses (§7.2, p.11: “missing a flag related to printing pandoc's highlighting style”), so there's a nearby, concrete case to extend into.

Opportunity: add format-representative fixtures (a small `.md`, `.csv`, and a minimal `.docx` / `.zip`) gated behind the existing `--content` level knob, so format-sensitive commands get proportionally richer coverage without inflating the fixture set for commands that don't need it.

8. No empty/zero-byte content variant [NOVEL]

All five `Content` values are non-empty (`ir/contents.py:11-26`). Empty input is one of the most common edge cases in real usage (`wc` on an empty file, `sort` on empty stdin, `head -c 0`), and is a frequent source of off-by-one or divide-by-zero-style bugs precisely because it's a boundary rather than a "typical" case.

Opportunity: add an `EMPTY` content variant — trivial to implement (an empty byte string) and cheap to run, since it doesn't add meaningfully to invocation count relative to the other content types already multiplied in.

9. LLM syntax-generation is evaluated once per command, without a reproducibility/variance check [NOVEL]

§7.2 (p.10-11) reports 99.7% correctness for the LLM-generated syntax specs, and §3.2 (p.5) describes a retry loop for *validation failures* (up to three attempts, with the previous output and error fed back). What isn't discussed is whether generating the *same* command's spec more than once (e.g., across several independent runs, or at a nonzero temperature) produces a consistently correct result, versus getting it right on the evaluated run and wrong on another. Since GPT-4o is not deterministic, this is a natural question for a number (99.7%) that downstream systems will treat as a reliability guarantee.

Opportunity: a repeated-sampling check on a subset of commands (e.g., 5 independent generations each) would turn the single-run 99.7% into a number with a reportable variance, strengthening the claim rather than changing it. **v2 relevance:** directly maps to comparison dimension 4 in `CLAUDE.md` ("Consistency/reproducibility across repeated runs — LLM nondeterminism is itself a result") — this is exactly the kind of check v2's evaluation harness should run for both the naive-LLM baseline *and*, retroactively, for v1's syntax-generation step.

Part 2 — Gaps already flagged by the authors

10. Symlinks are not part of the generated filesystem fixtures [PAPER-NOTED]

The filesystem model (`ir/environment.py:279-289`, `EnvironmentArgument = File | Directory | NonEmptyDirectory | NonexistentPath | ...`) has no symlink variant, so commands are never tested against a symlink target. The paper acknowledges this directly, in one sentence, right after presenting the model (§4.3, p.6): *“This model suffices for useful specifications (§7), though it can be extended (e.g., with symbolic links or pipes) to capture richer preconditions and effects.”* It is not listed in the paper’s formal “Limitations” paragraph (§2.2, p.4), and isn’t quantified anywhere. A concrete example that would benefit: the paper’s own §7.2 results note the LLM syntax parser *“omitted an option related to dereferencing command-line symlinks to directories for the `dir` command”* — a related but independent symlink gap, at the syntax-inference layer rather than the environment-generation layer.

Opportunity: add a `Symlink` environment shape (`symlink → file`, `symlink → directory`, and a dangling symlink) as a natural extension of the existing `LazyPath` hierarchy.

11. Pipe / early-reader-exit behavior (SIGPIPE) is untested [PAPER-NOTED]

Grouped with symlinks in the same sentence (§4.3, p.6: *“can be extended, e.g., with symbolic links or pipes”*), so the authors are aware of it — but it’s worth calling out on its own because of who v1’s primary downstream consumer is. PaSh (§6.1) exists specifically to parallelize *shell pipelines*, where a command’s stdout is frequently piped into another command that may exit early and close its read end. Whether a command handles `SIGPIPE` gracefully (vs. crashing, vs. silently losing writes) is exactly the kind of property a pipeline optimizer needs, and it isn’t observable from the single-process, non-piped invocations traced today.

Opportunity: since PaSh is the flagship consumer, a small number of piped-invocation traces (`command → head -n1`, to force early pipe closure) would directly strengthen the specification handed to its most prominent use case.

12. Numeric arguments are only ever -1, 0, or 1 [SELF-ASSESSED]

The paper states this directly: *“For integer, it generates the numbers -1, 0, 1”* (§4.2, p.6). This is a deliberate, disclosed choice, not a hidden one — but it means any command whose behavior depends on a numeric magnitude (`head -n 100`, `tail -c 4096`, `sleep 30`) is only ever exercised at the smallest possible boundary values, never at a “typical” or large value.

Opportunity: widen the integer generator to include one “typical” value (e.g. 10) and one “large” value, still bounded to keep the combinatorial cost the paper is optimizing for (§4.2’s whole point) manageable.

13. The 97.78% coverage figure blends normalization layers of different strength [SELF-ASSESSED]

§7.3 (p.11) is admirably transparent about this — it explicitly states that string normalization *“is not a valid normalization for commands whose semantics depend heavily on the input argument content (e.g., higher order commands like xargs...”* and that arity normalization is an *extrapolation* rather than an observation (*“if a command changes behavior depending on arity... Caruca wouldn’t be able to determine that”*). Both caveats are stated clearly by the authors. The opportunity is presentational: the final rollup (97.78%) combines exact matches, flag/path/integer normalization (all fairly strong), and string/arity normalization (explicitly weaker) into one number, which is the figure most likely to be quoted on its own.

Opportunity: reporting the “strong-normalization-only” subtotal (the paper’s own numbers put this around 18-19%, before string normalization is applied) alongside the full 97.78% would let readers see both the optimistic and conservative reads at a glance, matching the care already shown in the prose. **v2 relevance:** worth adopting this disaggregated-metric habit for v2’s own coverage numbers from the start (comparison dimension 3, “Coverage,” in `CLAUDE.md`), rather than retrofitting it later.

14. No environment-variable modeling anywhere in the generated invocation configurations [PAPER-NOTED]

The paper states this directly as one of only three items in its Limitations section (§2.2, p.4): *“Caruca does not generate environment variables in its invocation configurations, so it is agnostic to command behavior changes based on the values of environment variables.”* No environment-variable axis exists anywhere in the codebase’s configuration generation (`ir/environment.py`, `ir/string.py`) — invocation configs vary path type, string form, and (per items 6/8/12 above) permission/content/integer values, but never `$PATH`, locale variables (`LC_*`), or other env vars documented to change behavior.

Opportunity: add an environment-variable dimension to the configuration/fixture model — even a small set (a locale variable, a `PATH` variant, one command-specific variable for env-var-sensitive commands) would let commands whose documented behavior depends on environment state (many GNU coreutils honor `LC_ALL` / `LC_COLLATE` for sort order, for instance) get exercised along an axis the current model entirely omits. **v2 relevance:** a

natural addition to the v2-extended fixture profile already planned for Secure Sandbox ([ai_docs/prep/component_functionality.md](#)), alongside symlinks, permissions, and pipes — same “coverage beyond v1’s sweet spot” story (comparison dimension 3 in [CLAUDE.md](#)).

Summary table

#	GAP	LAYER	STATUS
1	No correlated multi-file content (diff/comm/join)	Codebase (generation strategy)	Novel
2	Parallelizability not validated under real concurrency	Methodology	Novel
3	Sandbox stack is Linux-only vs. cross-platform motivation	Methodology / scope	Novel
4	Real-world corpus limited to GitHub shell scripts	Evaluation corpus	Novel
5	Directories capped at one level, one child	Codebase (env model)	Novel
6	No permission/ownership axis	Codebase (env model)	Novel
7	No structured/office document content types	Codebase (content model)	Novel
8	No empty/zero-byte content variant	Codebase (content model)	Novel
9	No reproducibility check on LLM syntax generation	Evaluation methodology	Novel
10	No symlink fixtures	Codebase (env model)	Paper-noted
11	No pipe / SIGPIPE testing	Codebase (env model)	Paper-noted
12	Integers limited to {-1, 0, 1}	Codebase (arg generator)	Self-assessed
13	Aggregate coverage % blends strong and weak normalizations	Evaluation metric	Self-assessed
14	No environment-variable modeling	Codebase (env model)	Paper-noted